



MEDS LAB

Maktab-e-Digital Systems
Department of Electrical Engineering • UET Lahore

MODULE 3

RISC-V Instruction Set Architecture

RV32I Base Integer ISA • ABI • Assembly Programming • Privileged Architecture
Summer Training Programme 2026 • Cohort 4

Prepared by: Umer Shahid, Shehzeen Malik, & Aman Murad
Version 1.0 • May 2026



Table of Contents

Table of Contents.....	2
1. Module Overview & Compression Notes.....	4
2. Learning Objectives.....	5
Core ISA & Assembly Programming.....	5
Encoding & Architecture Awareness.....	5
3. Weekly Schedule.....	6
4. Day 1 — RISC-V Intro, Registers & Venus Tutorial.....	7
4.1 What is RISC-V?.....	7
Key Design Principles.....	7
4.2 The RISC-V Register File.....	7
4.3 Venus Simulator — Tutorial.....	8
4.3.1 Interface Overview.....	8
4.3.2 Your First Program.....	8
4.3.3 Execution Controls.....	9
4.3.4 Working with Memory (.data).....	9
4.3.5 Venus ECALL System Calls.....	9
4.4 Day 1 Exercises.....	10
5. Day 2 — Arithmetic, Logical & Memory Instructions.....	11
5.1 R-Type Instructions (Register-Register).....	11
5.2 I-Type Instructions (Immediate).....	11
5.3 LUI and AUIPC — Large Constants.....	11
5.4 Load Instructions.....	11
5.5 Store Instructions.....	12
5.6 Little-Endian Memory & Arrays.....	12
5.7 Day 2 Exercises.....	12
6. Day 3 — Control Flow: Branches, Jumps & Loops.....	14
6.1 Branch Instructions.....	14
6.2 Jump Instructions.....	14
6.3 Translating C to Assembly.....	14
If / Else.....	14
For Loop.....	14
While Loop.....	15
6.4 Pseudo-Instructions & Directives.....	15
Common Pseudo-Instructions.....	15
Assembler Directives.....	15
6.5 Day 3 Exercises.....	16
7. Day 4 — Functions, Calling Convention & Instruction Encoding.....	17
7.1 The Calling Convention.....	17



7.2 Stack Frame: Prologue & Epilogue.....	17
7.3 Complete Function Example.....	17
7.4 Recursion.....	18
7.5 Instruction Encoding (Crash Course).....	19
Encoding Example: add x4, x5, x10.....	19
Encoding Example: addi x2, x0, 5.....	19
7.6 Day 4 Exercises.....	20
8. Day 5 — Privileged Overview, Extensions & Grand Assignment.....	21
8.1 Privilege Levels (Conceptual).....	21
8.2 Key CSRs (Conceptual).....	21
Trap Handling Flow (High-Level).....	21
8.3 RISC-V Extensions (Overview).....	22
Profiles & Certification (RVA23).....	22
9. Grand Assignment: RISC-V Assembly Programming Challenge.....	23
9.1 Overview.....	23
9.2 Repository Structure.....	23
9.3 Part 1: Array Processing (30 points).....	23
9.4 Part 2: Recursive Algorithm (35 points).....	24
9.5 Part 3: Instruction Encoding (15 points).....	24
9.6 Self-Study Deliverables (20 points).....	24
9.7 Bonus (+10 points).....	24
10. Assessment Rubric.....	25
11. Recommended Resources.....	26
Specifications.....	26
Simulator.....	26
Courses & Books.....	26
Practice.....	26
12. Appendix: RV32I Reference Card & ABI Table.....	27
A. Complete RV32I Instruction List.....	27
B. ABI Register Names.....	27
C. Calling Convention.....	27
D. Instruction Encoding Quick Reference.....	27



1. Module Overview & Compression Notes

Module 3 covers the RISC-V Instruction Set Architecture — the contract between hardware and software that underpins everything at MEDS Lab. You will learn to read, write, and debug RISC-V assembly, understand the hardware-software interface, and reason about processor behaviour at the instruction level.

Venus RISC-V Simulator

Throughout this module, you will use Venus (<https://venus.cs61c.org/>) — a web-based RISC-V simulator from UC Berkeley. Venus lets you write, assemble, step through, and debug RISC-V programs directly in your browser. A full Venus tutorial is included on Day 1.

Pre-requisites: Modules 1 & 2 completed. Familiarity with binary/hex, bitwise operations, and C.

Primary Reference: RISC-V Unprivileged Specification (Volume 1), Chapters 1–2.

Simulator: Venus — <https://venus.cs61c.org/> (web-based, no installation needed)

Deliverable: Grand Assignment — 3-part RISC-V assembly challenge submitted via GitHub.



2. Learning Objectives

By the end of this module, every mentee should be able to:

Core ISA & Assembly Programming

- Explain the RISC-V design philosophy and how it differs from x86 and ARM
- List all 32 general-purpose registers and their ABI names (x0–x31 / zero, ra, sp, ..., t6)
- Write RISC-V assembly programs using arithmetic, logical, shift, and comparison instructions
- Use load/store instructions (LW, LB, SW, SB) and understand byte-addressable, little-endian memory
- Implement branching logic (BEQ, BNE, BLT, BGE) and loops in assembly
- Use JAL/JALR for function calls and implement the RISC-V calling convention
- Build and manage a stack frame: prologue, epilogue, argument passing, return values
- Use Venus to write, assemble, step through, and debug RISC-V programs
- Use pseudo-instructions (LI, LA, MV, NOP, RET, CALL) and assembler directives (.data, .text, .word)

Encoding & Architecture Awareness

- Manually encode and decode instructions in all 6 RISC-V formats (R, I, S, B, U, J)
- Explain the RISC-V privilege levels (M, S, U) and key CSRs at a conceptual level
- Describe the trap/interrupt mechanism at a high level
- List RISC-V standard extensions (M, A, F, D, C, V) and understand the profile concept (RVA23)



3. Weekly Schedule

The module covers 5 days. The pace is aggressive, come prepared, review each evening.

Day	Topics	Mode
Day 1	RISC-V intro, design philosophy, register file (ABI), Venus simulator tutorial	Lab
Day 2	Arithmetic, logical, shift, immediate instructions + loads, stores, memory, arrays	Lab
Day 3	Branches, jumps, loops, if/else in assembly + pseudo-instructions & directives	Lab
Day 4	Functions, calling convention, stack frames, recursion + instruction encoding	Lab
Day 5	Privileged architecture overview, extensions/profiles, Grand Assignment start	Lab

Self-Study Required: ~5 hours additional reading (privileged spec, extensions) assigned as homework

Grand Assignment: Briefed on Day 4 afternoon, worked on Day 5, due 48 hours after Day 5 session.



4. Day 1 — RISC-V Intro, Registers & Venus Tutorial

Goal: Understand what RISC-V is, learn the register file, and become proficient with the Venus simulator.

4.1 What is RISC-V?

RISC-V (pronounced “risk-five”) is an open-standard instruction set architecture based on RISC principles. Created at UC Berkeley in 2010, now governed by RISC-V International with 4,000+ members worldwide.

Key Design Principles

- **Simplicity:** A small, clean base ISA (RV32I has only 47 instructions) — easy to implement in hardware
- **Modularity:** Standard extensions (M for multiply, F for float, V for vector) added as needed
- **Openness:** No licensing fees, no royalties — anyone can build a RISC-V processor
- **Stability:** The base ISA is frozen — software written today runs forever
- **Extensibility:** Custom extensions for domain-specific acceleration

Comparison: x86 has 1,000+ instructions and is proprietary (Intel/AMD). ARM is licensed and moderately complex. RISC-V is open, simple, and growing rapidly in embedded, IoT, and datacenter applications.

Why MEDS Focuses on RISC-V

RISC-V’s openness means you can read the spec, build a processor, write tests, and contribute to the ecosystem — all without NDAs or license agreements. The certification test suite at RISC-V International, open-source processor designs, and the entire MEDS Lab workflow builds on RISC-V.

4.2 The RISC-V Register File

RV32I has 32 general-purpose registers, each 32 bits wide. Register x0 is hardwired to zero. The ABI assigns conventional names and roles:

Reg	ABI	Saver	Role	Description
x0	zero	—	Hardwired zero	Always reads as 0; writes ignored
x1	ra	Caller	Return address	Holds return address after JAL/JALR
x2	sp	Callee	Stack pointer	Points to top of stack; must be 16-byte aligned



x3	gp	—	Global pointer	Base for global data access
x4	tp	—	Thread pointer	Thread-local storage pointer
x5–x7	t0–t2	Caller	Temporaries	Caller-saved; free to clobber in functions
x8	s0/fp	Callee	Saved / Frame ptr	Callee-saved; optionally frame pointer
x9	s1	Callee	Saved register	Callee-saved
x10–x11	a0–a1	Caller	Args / Return	Function arguments and return values
x12–x17	a2–a7	Caller	Arguments	Additional function arguments
x18–x27	s2–s11	Callee	Saved registers	Callee-saved; preserved across calls
x28–x31	t3–t6	Caller	Temporaries	Caller-saved; free to use

Caller-Saved vs. Callee-Saved

Caller-saved (t0–t6, a0–a7, ra): The CALLING function must save these before calling, because the callee may overwrite them. Callee-saved (s0–s11, sp): The CALLED function must save and restore these before returning. This convention makes function calls predictable.

4.3 Venus Simulator — Tutorial

Venus runs at <https://venus.cs61c.org/> — no installation needed. This section covers everything you need.

Open Venus Now

Navigate to <https://venus.cs61c.org/> in your browser. Bookmark it — you will use it every day this week.

4.3.1 Interface Overview

- Editor tab: Write RISC-V assembly with syntax highlighting.
- Simulator tab: Run/step/debug your program after assembling.

Click the Simulator tab (or Assemble & Simulate) to assemble your code and switch to the simulator view.

4.3.2 Your First Program

```
# My first RISC-V program – adds two numbers
.text
.globl main
main:
    addi a0, zero, 5      # a0 = 5
    addi a1, zero, 10     # a1 = 10
    add  a2, a0, a1       # a2 = 15
    # Print result
```




```

addi a0, zero, 1      # ecall 1 = print int
mv   a1, a2           # a1 = value to print
ecall
# Exit
addi a0, zero, 10     # ecall 10 = exit
ecall

```

4.3.3 Execution Controls

- Step (▶): Execute ONE instruction. Watch registers change. This is your most-used button.
- Run (▶▶): Run the entire program to completion.
- Prev (◀): Step BACKWARD. Invaluable for debugging.
- Reset: Reset PC to beginning, clear all changes.

Try It Now

Step through your first program. After step 1: a0=5. After step 2: a1=10. After step 3: a2=15. Verify each in the Registers panel on the right.

4.3.4 Working with Memory (.data)

```

.data
my_array: .word 10, 20, 30, 40, 50
message:  .string "Hello MEDS!"

.text
main:
    la    t0, my_array      # t0 = address of my_array
    lw    t1, 0(t0)         # t1 = 10
    lw    t2, 4(t0)         # t2 = 20
    add   t3, t1, t2        # t3 = 30
    addi  a0, zero, 10
    ecall

```

Check the Memory panel (Data segment) to see your array values in little-endian byte order.

4.3.5 Venus ECALL System Calls

a0 Value	Action	Argument	Result
1	Print integer	a1 = integer	Prints to console
4	Print string	a1 = address	Prints null-terminated string
5	Read integer	—	Result in a0



10	Exit program	—	Terminates
11	Print character	a1 = char	Prints ASCII char
17	Exit with code	a1 = exit code	Terminates with code

4.4 Day 1 Exercises

Day 1 Hands-On Exercises (2 hours)

- ▶ Exercise 1: Write a Venus program that loads 42 into a0 and 58 into a1, adds them, and prints the result (100) using ecall 1. Step through every instruction.
- ▶ Exercise 2: Write a program that reads two integers from user (ecall 5), adds them, and prints the sum.
- ▶ Exercise 3: Write a program that prints "Hello MEDS!" using a .data string and ecall 4.
- ▶ Exercise 4: Explore all 32 registers: write a unique value to every register (except x0). Confirm x0 always reads 0.
- ▶ Exercise 5: Write a program that computes $1+2+3+\dots+N$ for a user-provided N using a loop. Print the result.
- ▶ Exercise 6 (Bonus): Write a program that reads a number and prints whether it is even or odd (use ANDI with 1).

5. Day 2 — Arithmetic, Logical & Memory Instructions

Goal: Master all RV32I compute instructions AND load/store memory access in a single day.

5.1 R-Type Instructions (Register-Register)

```
# Syntax: INSTR rd, rs1, rs2
add  t0, t1, t2    # t0 = t1 + t2
sub  t0, t1, t2    # t0 = t1 - t2
and  t0, t1, t2    # t0 = t1 & t2  (bitwise AND)
or   t0, t1, t2    # t0 = t1 | t2  (bitwise OR)
xor  t0, t1, t2    # t0 = t1 ^ t2  (bitwise XOR)
sll  t0, t1, t2    # t0 = t1 << t2 (shift left logical)
srl  t0, t1, t2    # t0 = t1 >> t2 (shift right logical, zero-fill)
sra  t0, t1, t2    # t0 = t1 >> t2 (shift right arithmetic, sign-fill)
slt  t0, t1, t2    # t0 = (t1 < t2) ? 1 : 0  (signed)
sltu t0, t1, t2    # t0 = (t1 < t2) ? 1 : 0  (unsigned)
```

5.2 I-Type Instructions (Immediate)

Use a 12-bit signed immediate (−2048 to +2047) instead of a second register:

```
addi t0, t1, 5      # t0 = t1 + 5    (NO subi – use negative imm)
andi t0, t1, 0xFF   # Mask lower byte
ori  t0, t1, 0x80   # Set bit 7
xori t0, t1, -1     # Bitwise NOT (XOR with all 1s)
slti t0, t1, 100    # t0 = (t1 < 100) ? 1 : 0
slli t0, t1, 2      # Multiply by 4
srli t0, t1, 4      # Unsigned divide by 16
srai t0, t1, 4      # Signed divide by 16
```

5.3 LUI and AUIPC — Large Constants

```
lui  t0, 0x12345    # t0 = 0x12345000 (upper 20 bits)
addi t0, t0, 0x678   # t0 = 0x12345678
# The LI pseudo-instruction does this automatically:
li   t0, 0x12345678  # Expands to LUI + ADDI
auipc t0, 0x10       # t0 = PC + 0x10000
```

5.4 Load Instructions

RISC-V is a load/store architecture: ONLY load/store instructions access memory. All arithmetic operates on registers.

```
# Syntax: LOAD rd, offset(rs1) - rd = Mem[rs1 + offset]
lw  t0, 0(t1)      # Load Word (32 bits)
lh  t0, 0(t1)      # Load Half (16 bits, sign-extended)
lhu t0, 0(t1)      # Load Half Unsigned (16 bits, zero-extended)
lb  t0, 0(t1)      # Load Byte (8 bits, sign-extended)
lbu t0, 0(t1)      # Load Byte Unsigned (8 bits, zero-extended)
```

5.5 Store Instructions

```
# Syntax: STORE rs2, offset(rs1) - Mem[rs1 + offset] = rs2
sw  t0, 0(t1)      # Store Word (32 bits)
sh  t0, 0(t1)      # Store Half (16 bits)
sb  t0, 0(t1)      # Store Byte (8 bits)
```

5.6 Little-Endian Memory & Arrays

RISC-V uses little-endian: the least significant byte is at the lowest address. Storing 0xDEADBEEF at address 0x1000 places EF at 0x1000, BE at 0x1001, AD at 0x1002, DE at 0x1003.

```
.data
array: .word 10, 20, 30, 40, 50

.text
main:
    la    s0, array      # s0 = base address
    li    s1, 5           # array size
    li    s2, 0           # sum = 0
    li    t0, 0           # i = 0
loop:
    bge   t0, s1, done    # if i >= size, exit
    slli  t1, t0, 2        # t1 = i * 4 (byte offset)
    add   t2, s0, t1       # t2 = &array[i]
    lw    t3, 0(t2)        # t3 = array[i]
    add   s2, s2, t3       # sum += array[i]
    addi  t0, t0, 1        # i++
    j     loop
done:
    # s2 = 150
```

5.7 Day 2 Exercises



Day 2 Hands-On Exercises (2 hours)

- ▶ Exercise 1: Write a program that computes $(b * 8) - (c / 4)$ using ONLY shifts and add/sub. Let $b=12$, $c=64$. Expected: 80.
- ▶ Exercise 2: Write a program that extracts the lower byte, second byte, and upper half-word from 0xDEADBEEF using AND/shift. Print each.
- ▶ Exercise 3: Create a .data array of 8 words. Write a program that computes and prints the sum. Verify in Venus memory panel.
- ▶ Exercise 4: Write a program that finds the maximum value in a .data array of 10 signed integers (include negatives).
- ▶ Exercise 5: Store 0xDEADBEEF in .data. Load it as word, half-word (at offset 0 and 2), and byte. Print each to verify endianness.

6. Day 3 — Control Flow: Branches, Jumps & Loops

Goal: Implement if/else, loops, and unconditional jumps. Learn pseudo-instructions and assembler directives.

6.1 Branch Instructions

```
beq  t0, t1, label    # Branch if t0 == t1
bne  t0, t1, label    # Branch if t0 != t1
blt  t0, t1, label    # Branch if t0 < t1 (signed)
bge  t0, t1, label    # Branch if t0 >= t1 (signed)
bltu t0, t1, label    # Branch if t0 < t1 (unsigned)
bgeu t0, t1, label    # Branch if t0 >= t1 (unsigned)
```

6.2 Jump Instructions

```
jal  rd, label    # rd = PC+4; PC = label (function call)
jalr rd, rs1, imm # rd = PC+4; PC = rs1+imm (return / indirect)
```

Pseudo-instructions:

```
j    label        # Unconditional jump (jal zero, label)
jr   rs1          # Jump register (jalr zero, rs1, 0)
ret                     # Return (jalr zero, ra, 0)
call label        # Far call (auipc+jalr)
```

6.3 Translating C to Assembly

If / Else

```
# C: if (a == b) { x = 1; } else { x = 2; }
    bne  t0, t1, else_    # if a != b, go to else
    addi t2, zero, 1      # x = 1
    j    endif_
else_:
    addi t2, zero, 2      # x = 2
endif_:
```

For Loop

```
# C: for (int i = 0; i < 10; i++) { sum += i; }
    li   t0, 0            # i = 0
    li   t1, 0            # sum = 0
    li   t2, 10           # limit
for_:
```

```

    bge t0, t2, endfor_   # if i >= 10, exit
    add t1, t1, t0        # sum += i
    addi t0, t0, 1        # i++
    j    for_
endfor_:                  # t1 = 45

```

While Loop

```

# C: while (n > 0) { count++; n >= 1; }
    li    t1, 0          # count = 0
while_:
    blez t0, endwhile_   # if n <= 0, exit (pseudo)
    addi t1, t1, 1
    srli t0, t0, 1
    j     while_
endwhile_:

```

6.4 Pseudo-Instructions & Directives

Common Pseudo-Instructions

```

li    rd, imm    # Load immediate (lui+addi)
la    rd, label   # Load address  (auipc+addi)
mv    rd, rs      # Move register  (addi rd, rs, 0)
nop                    # No operation  (addi zero, zero, 0)
not   rd, rs      # Bitwise NOT      (xori rd, rs, -1)
neg   rd, rs      # Negate           (sub rd, zero, rs)
beqz  rs, label   # Branch if zero  (beq rs, zero, label)
bnez  rs, label   # Branch if !0    (bne rs, zero, label)
bgt   rs,rt,label # Branch if >     (blt rt, rs, label)
ble   rs,rt,label # Branch if <=    (bge rt, rs, label)

```

Assembler Directives

```

.text                # Code section
.data                # Data section
.word  42, 100, -5   # 32-bit values
.half  0xABCD        # 16-bit value
.byte  0xFF, 0x00    # 8-bit values
.string "Hello"      # Null-terminated string
.globl main          # Make symbol visible to linker
.align 2             # Align to 4-byte boundary
.equ   SIZE, 100     # Define constant

```



6.5 Day 3 Exercises

Day 3 Hands-On Exercises (2 hours)

- ▶ Exercise 1: Write a program that reads N and prints whether it is positive, negative, or zero.
- ▶ Exercise 2: Implement iterative factorial: read N , compute $N!$, print result.
- ▶ Exercise 3: Write a program that reverses a .data array in-place using swap logic. Print reversed array.
- ▶ Exercise 4: Implement binary search on a sorted .data array. Print index or -1 if not found.
- ▶ Exercise 5: Step through a program with pseudo-instructions in Venus. For each pseudo, note what real instruction(s) it expands to in the instruction panel.
- ▶ Exercise 6 (Bonus): Implement bubble sort entirely in assembly. Print sorted array.

7. Day 4 — Functions, Calling Convention & Instruction Encoding

Goal: Write proper functions with stack frames, follow the calling convention, implement recursion. Understand instruction encoding.

7.1 The Calling Convention

- Arguments: a0–a7 (up to 8 args; extras on stack)
- Return value: a0
- Return address: ra (set by JAL/CALL)
- Caller-saved: t0–t6, a0–a7, ra
- Callee-saved: s0–s11, sp

7.2 Stack Frame: Prologue & Epilogue

```
my_function:
    # PROLOGUE – save callee-saved regs + ra
    addi sp, sp, -16      # Allocate 16 bytes
    sw   ra, 12(sp)       # Save return address
    sw   s0, 8(sp)        # Save s0 if we use it

    # ... function body ...

    # EPILOGUE – restore and return
    lw   s0, 8(sp)
    lw   ra, 12(sp)
    addi sp, sp, 16       # Deallocate
    ret
```

Stack Must Be 16-Byte Aligned

The RISC-V ABI requires sp to be 16-byte aligned. Always allocate in multiples of 16.

7.3 Complete Function Example

```
# int multiply(int a, int b) – repeated addition
multiply:                                # Leaf function: no call, no save ra
    li   t0, 0                          # result = 0
    blez a1, mul_done
```

```

mul_loop:
    add  t0, t0, a0
    addi a1, a1, -1
    bnez a1, mul_loop
mul_done:
    mv   a0, t0                # return result
    ret

main:
    addi sp, sp, -16
    sw   ra, 12(sp)
    li   a0, 7
    li   a1, 6
    call multiply              # a0 = 42
    mv   a1, a0
    li   a0, 1
    ecall                      # prints 42
    lw   ra, 12(sp)
    addi sp, sp, 16
    li   a0, 10
    ecall

```

7.4 Recursion

```

# int factorial(int n) {
#     if (n <= 1) return 1;
#     return n * factorial(n-1);
# }

factorial:
    addi sp, sp, -16
    sw   ra, 12(sp)
    sw   s0, 8(sp)
    mv   s0, a0                # s0 = n (preserved across call)
    li   t0, 1
    ble  a0, t0, base_case
    addi a0, s0, -1            # a0 = n-1
    call factorial             # a0 = factorial(n-1)
    mv   a1, a0
    mv   a0, s0
    call multiply              # a0 = n * factorial(n-1)
    j    fact_ret

```



```
base_case:
    li    a0, 1
fact_ret:
    lw    s0, 8(sp)
    lw    ra, 12(sp)
    addi  sp, sp, 16
    ret
```

7.5 Instruction Encoding (Crash Course)

Every RV32I instruction is exactly 32 bits. The 6 formats determine how bits are divided:

```
R: | funct7[31:25] | rs2[24:20] | rs1[19:15] | f3[14:12] | rd[11:7] | opcode[6:0] |
I: |      imm[31:20]      | rs1[19:15] | f3[14:12] | rd[11:7] | opcode[6:0] |
S: | imm[11:5]      | rs2[24:20] | rs1[19:15] | f3[14:12] | imm[4:0] | opcode[6:0] |
B: | imm[12:10:5]
   | rs2[24:20] | rs1[19:15] | f3[14:12] | imm[4:1:11] | opcode[6:0] |
U: |      imm[31:12]      | rd[11:7] | opcode[6:0] |
J: |      imm[20:10:1:11:19:12]      | rd[11:7] | opcode[6:0] |
```

Encoding Example: add x4, x5, x10

```
R-type: opcode=0110011, funct3=000, funct7=0000000
rd=x4=00100, rs1=x5=00101, rs2=x10=01010
```

```
Binary: 00000000 01010 00101 000 00100 0110011
Hex: 0x00A28233
```

Encoding Example: addi x2, x0, 5

```
I-type: opcode=0010011, funct3=000, imm=0000000000101
rd=x2=00010, rs1=x0=00000
```

```
Binary: 0000000000101 00000 000 00010 0010011
Hex: 0x00500113
```

Hardware Connection

In Module 2, you wrote a C decoder using shift-and-mask for these exact bit positions. When you design RTL in Modules 4–5, you will wire assign opcode = instruction[6:0] — the same fields.



7.6 Day 4 Exercises

Day 4 Hands-On Exercises (2 hours)

- ▶ Exercise 1: Write `max(a0,a1) → a0` and `sum_array(a0=ptr, a1=size) → a0` functions. Call both from main with test data.
- ▶ Exercise 2: Implement recursive Fibonacci: `fib(0)=0`, `fib(1)=1`, `fib(n)=fib(n-1)+fib(n-2)`. Print `fib(10)=55`.
- ▶ Exercise 3: Write a function that is called by another function (nested calls). Step through in Venus to verify ra and s-registers are properly saved/restored.
- ▶ Exercise 4: Hand-encode these instructions to hex: `sub x1,x2,x3` | `ori x5,x6,0xFF` | `sw x7,8(x8)` | `beq x1,x2,+16`. Verify against Venus.
- ▶ Exercise 5: Decode these hex values to assembly: `0x00A28233` | `0x00500113` | `0xFE209CE3`. Verify in Venus.
- ▶ Exercise 6 (Bonus): Implement recursive `power(base, exp)` returning `base^exp`. Test: `power(2,10)=1024`.



8. Day 5 — Privileged Overview, Extensions & Grand Assignment

Goal: Gain conceptual understanding of privileged architecture and extensions. Begin the Grand Assignment.

8.1 Privilege Levels (Conceptual)

- Machine Mode (M-mode): Highest privilege. Always present. Firmware/bootloader runs here.
- Supervisor Mode (S-mode): Optional. OS kernel runs here. Manages virtual memory.
- User Mode (U-mode): Optional. Applications run here. Restricted hardware access.

A minimal RISC-V core (like your Module 5 design) may only have M-mode. A Linux system needs all three.

8.2 Key CSRs (Conceptual)

- mstatus: Global interrupt enable, privilege mode tracking
- mtvec: Trap handler address — where processor jumps on exception/interrupt
- mepc: Exception PC — address of instruction that caused the trap
- mcause: Trap cause code — identifies what happened (illegal instruction, ecall, timer, etc.)
- mtval: Additional trap info (e.g., faulting address)

Trap Handling Flow (High-Level)

1. Processor detects trap (exception or interrupt)
2. Hardware: mepc = current PC, mcause = cause code, PC → mtvec
3. Trap handler executes, reads mcause, handles the situation
4. Handler executes MRET: PC = mepc, execution resumes

Self-Study Assignment

Read Sections 3.1–3.4 of the RISC-V Privileged Specification (Volume 2). Focus on understanding the CSR table and the trap handling flow. Write a one-page summary and submit with your Grand Assignment.



8.3 RISC-V Extensions (Overview)

Ext	Name	What It Adds
M	Multiplication	MUL, DIV, REM — integer multiply/divide
A	Atomic	LR/SC, AMO — for multicore synchronization
F	Float (single)	32-bit floating-point + f0–f31 registers
D	Float (double)	64-bit floating-point
C	Compressed	16-bit instructions — ~25% code size reduction
V	Vector	Scalable vector operations for SIMD/ML workloads

Profiles & Certification (RVA23)

A profile bundles base ISA + extensions that an implementation must support. The RVA23 profile defines what a modern application processor needs for Linux. MEDS Lab contributes directly to the RISC-V International certification test suite that validates implementations against these profiles.

Self-Study Assignment

Pick one extension (V, A, or C) and read its chapter in the RISC-V spec. Write a half-page summary of what it adds and why it matters. Submit with your Grand Assignment.



9. Grand Assignment: RISC-V Assembly Programming Challenge

GRAND ASSIGNMENT

MEDS Module 3 — Due: 48 Hours After Day 5

Submission: GitHub repo with .s files, README, Venus screenshots, self-study summaries

9.1 Overview

This assignment consists of 3 assembly programs plus 2 written deliverables. Each program must run correctly in Venus, follow the RISC-V calling convention, and be well-commented.

9.2 Repository Structure

```
riscv-assembly-challenge/
├── README.md           # Overview, build/run instructions
├── .gitignore
├── part1_array_ops.s    # Part 1: Array processing
├── part2_recursion.s    # Part 2: Recursive algorithms
├── part3_encoding.s     # Part 3: Instruction decoder in asm
├── screenshots/        # Venus screenshots showing output
├── docs/
│   ├── ENCODING_WORKSHEET.md # Hand-encoded instructions
│   ├── PRIVILEGED_SUMMARY.md # Privileged spec self-study
│   └── EXTENSION_SUMMARY.md  # Extension self-study
```

9.3 Part 1: Array Processing (30 points)

Write a single program (part1_array_ops.s) that:

- Defines a .data array of at least 12 signed integers (include negative numbers)
- Implements these functions, each taking (a0=array_ptr, a1=size) and returning result in a0:
 - sum_array — returns the sum of all elements
 - find_min — returns the minimum value (signed)
 - find_max — returns the maximum value (signed)
 - count_negative — returns how many elements are negative
- Main calls each function, prints results with labels (e.g., "Sum: 42")



- All functions must follow calling convention (save/restore callee-saved regs)

9.4 Part 2: Recursive Algorithm (35 points)

Implement ONE of the following in `part2_recursion.s`:

- Option A: Recursive merge sort on a `.data` array. Print the sorted result.
- Option B: Tower of Hanoi for N disks. Print each move (e.g., "Move disk 1 from A to C").
- Option C: Recursive Fibonacci with memoization (use a `.data` cache array to avoid recomputation). Print `fib(20)`.

Requirements:

- Must use genuine recursion (not loops disguised as recursion)
- Must follow calling convention perfectly — this is the primary grading criterion
- Must work for at least 3 different inputs
- Include Venus screenshots showing correct output for each test case

9.5 Part 3: Instruction Encoding (15 points)

- Hand-encode 6 instructions (one of each format: R, I, S, B, U, J) in `ENCODING_WORKSHEET.md`. Show all work.
- Write `part3_encoding.s`: load these 6 hex values as `.word` data. For each, extract and print `opcode`, `rd`, `rs1`, `funct3` using shift-and-mask operations.

9.6 Self-Study Deliverables (20 points)

- `PRIVILEGED_SUMMARY.md` (10 pts): One-page summary of RISC-V privileged architecture covering privilege levels, key CSRs, and trap handling flow. Based on reading Privileged Spec Sections 3.1–3.4.
- `EXTENSION_SUMMARY.md` (10 pts): Half-page summary of one RISC-V extension (V, A, or C) covering what it adds, key instructions, and practical applications.

9.7 Bonus (+10 points)

- Implement TWO recursive algorithms instead of one (+5)
- In Part 3, print actual instruction mnemonics (not just field values) — a mini disassembler in assembly (+3)
- In Part 1, add a `selection_sort` function and print the sorted array (+2)



10. Assessment Rubric

Component	Points	Criteria
Part 1: Array Ops	30	4 functions work, calling convention followed, results printed with labels, callee-saved regs preserved
Part 2: Recursion	35	Genuine recursion, stack frames correct, calling convention perfect, works for 3+ inputs, screenshots
Part 3: Encoding	15	6 instructions hand-encoded correctly (worksheet), Venus extractor works, fields printed correctly
Self-Study Docs	20	Privileged summary accurate and complete (10), extension summary insightful (10)
Bonus	+10	2nd recursion algo (+5), disassembler (+3), sort function (+2)
TOTAL	100+10	

Grading: 90–100: Exceptional | 75–89: Proficient | 60–74: Developing | Below 60: Needs Rework



11. Recommended Resources

Specifications

- RISC-V Unprivileged Spec (Volume 1): <https://riscv.org/technical/specifications/>
- RISC-V Privileged Spec (Volume 2): Same link — required for self-study deliverable
- RISC-V Reference Card: <https://github.com/jameslzh/riscv-card> — Print this out!

Simulator

- Venus: <https://venus.cs61c.org/> — Your primary tool
- RARS: <https://github.com/TheThirdOne/rars> — Alternative with more features

Courses & Books

- CS 61C (UC Berkeley): Lectures 5–15 on RISC-V assembly
- Computer Organization and Design, RISC-V Edition (Patterson & Hennessy) — THE textbook
- MIT 6.004 — Computation Structures

Practice

- RISC-V Jump Start: <https://projectf.io/posts/riscv-jump-start/>
- Learn by translating: take C programs from Module 2 and rewrite them in assembly



12. Appendix: RV32I Reference Card & ABI Table

A. Complete RV32I Instruction List

```
# Arithmetic:  ADD SUB ADDI  SLT SLTU SLTI SLTIU
# Logical:     AND OR XOR  ANDI ORI XORI
# Shift:      SLL SRL SRA  SLLI SRLI SRAI
# Upper Imm:   LUI AUIPC
# Load:      LW LH LHU LB LBU
# Store:      SW SH SB
# Branch:     BEQ BNE BLT BGE BLTU BGEU
# Jump:      JAL JALR
# System:     ECALL EBREAK FENCE
# Total: 47 instructions in RV32I base
```

B. ABI Register Names

```
x0=zero  x8=s0/fp  x16=a6   x24=s8
x1=ra    x9=s1    x17=a7   x25=s9
x2=sp    x10=a0   x18=s2   x26=s10
x3=gp    x11=a1   x19=s3   x27=s11
x4=tp    x12=a2   x20=s4   x28=t3
x5=t0    x13=a3   x21=s5   x29=t4
x6=t1    x14=a4   x22=s6   x30=t5
x7=t2    x15=a5   x23=s7   x31=t6
```

C. Calling Convention

Arguments:	a0-a7 (x10-x17)	Caller-saved
Return:	a0 (x10)	Caller-saved
Return addr:	ra (x1)	Caller-saved
Stack ptr:	sp (x2)	Callee-saved, 16B aligned
Saved regs:	s0-s11	Callee-saved
Temp regs:	t0-t6	Caller-saved

Prologue: `addi sp,sp,-N; sw ra,N-4(sp); sw s0,N-8(sp);...`

Epilogue: `lw s0,N-8(sp); lw ra,N-4(sp); addi sp,sp,N; ret`

D. Instruction Encoding Quick Reference

```
R: |f7(7)|rs2(5)|rs1(5)|f3(3)|rd(5)|op(7)|  op=0110011
I: |imm(12)|rs1(5)|f3(3)|rd(5)|op(7)|  op=0010011/0000011
```



```
S: |imm[11:5](7)|rs2(5)|rs1(5)|f3(3)|imm[4:0](5)|op(7)| op=0100011
B: |imm(7)          |rs2(5)|rs1(5)|f3(3)|imm(5)          |op(7)| op=1100011
U: |imm(20)          |rd(5)|op(7)| op=0110111/0010111
J: |imm(20)          |rd(5)|op(7)| op=1101111
```

End of Module 3

MEDS Lab • UET Lahore • Summer 2026

"The ISA is the contract between hardware and software. You now speak both languages."